

CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Speaker: Kyle Sha
(Simulation-Based Engineering Lab)

One loop, three variables

- Every human-in-the-loop simulation in this tutorial is the same loop:
 - read three numbers from the human -> push them into a model -> step it -> wait for real time -> send some numbers back
- What changes from part to part is not the loop. It is three variables around it:
 - WHERE THE NUMBERS COME FROM - a keyboard, a wheel, a socket, another machine
 - WHAT THEY DRIVE - a Chrono::Vehicle, a Viper rover, a gantry crane with a swinging load
 - WHERE IT HAPPENS - a 200 x 200 m patch, or the Mcity digital twin
- The invariant is worth more than any of the variables, so we start there and then change one thing at a time

Before we start

- This is a walkthrough - nothing to install right now; everything runs on my machine
- Code, slides and a written walkthrough: github.com/ksha23/chrono-hil-tutorial
- To run it yourself afterward:
 - `conda create -n chrono_hil -c projectchrono -c conda-forge pychrono pygame python=3.13`
 - `conda activate chrono_hil`
- Do NOT `pip install pychrono` - the PyPI package with that name is an unrelated library
- `python tutorial_HIL_driver.py` opens an Irrlicht window with an HMMWV - drive it with W/A/S/D
- All switches are in the CONFIGURATION section at the bottom of `tutorial_HIL_driver.py`

The invariant: one loop

The loop, and where the driver is queried (Lines 700 to 751)

```
while vis.Run():
    sim_time = system.GetChTime()

    # Render scene
    if step_number % render_steps == 0:
        follow_camera()
        vis.BeginScene()
        vis.Render()
        vis.EndScene()

    # PART 4 samples the device here; PART 6 applies any gear command
    ...

    # Get driver inputs (three floats) - the whole human-in-the-loop contract
    driver_inputs = driver.GetInputs()

    # PART 8: hand the three numbers to whatever is being controlled
    plant.apply(driver_inputs)

    # Update modules (process inputs from other modules)
    driver.Synchronize(sim_time)
    terrain.Synchronize(sim_time)
    plant.synchronize(sim_time, driver_inputs)
    vis.Synchronize(sim_time, driver_inputs)

    # Advance simulation for one timestep for all modules
    driver.Advance(step_size)
    terrain.Advance(step_size)
    plant.advance(step_size) # spins here if REALTIME == "vehicle"
    system.DoStepDynamics(step_size) # only for a plant that does not step itself
    vis.Advance(step_size)
```

Synchronize: every module reads what it needs from the others at time t . Advance: every module integrates t to $t + \text{step}$
Nothing below while `vis.Run()`: branches on which part of the tutorial you are on. That is the point of this slide

What Chrono actually requires of a human interface (Lines 151 to 178)



```
class DriverInputs:
    """The whole ChDriver contract, in Python."""

    def __init__(self):
        self.m_steering = 0.0
        self.m_throttle = 0.0
        self.m_braking = 0.0

    def SetSteering(self, v):
        self.m_steering = v

    def SetThrottle(self, v):
        self.m_throttle = v

    def SetBraking(self, v):
        self.m_braking = v

    def GetInputs(self):
        return self
```

Three floats per step: steering $[-1, 1]$, throttle $[0, 1]$, braking $[0, 1]$

`veh.ChDriver` is the same container in C++; this Python stand-in is what the rover and the crane use

That is the entire contract. Everything else in this talk is about filling it in, on time, from somewhere real

What "real-time" means in Chrono

- Chrono integrates as fast as it can; it has no notion of wall-clock time by itself
- Real Time Factor: $RTF = \text{compute time for a step} / \text{step size}$
 - $RTF < 1$: faster than real time - we must WAIT after each step
 - $RTF > 1$: slower than real time - nothing can be done except a cheaper model or a bigger step
- Soft real-time: spin after each step until wall time catches up with simulation time
 - No OS scheduling guarantees; good enough for a driving simulator, not for a control ECU
- Three equivalent mechanisms in PyChrono: `ChRealtimeStepTimer`, `ChVehicle.EnableRealtime`, or your own

Enforce real time, and measure the drift (Lines 657 to 671, 727 to 743)



```
# Real-time setup (PART 1)
realtime_mode = REALTIME
if REALTIME == "vehicle":
    if vehicle:
        vehicle.EnableRealtime(True)
    else:
        # PART 8: there is no vehicle to do the spinning, so fall back to
        # the timer that does the same thing from outside.
        realtime_mode = "per_step"
rt_timer = chrono.ChRealtimeStepTimer() # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()   # used when REALTIME == "cumulative"

# ... and once per simulated second, the report that tells you if it worked:

# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    ...
    print(f"{sim_time:7.2f} {wall:7.2f} {wall - sim_time:+7.3f} {rtf:6.2f}")
```

"vehicle" spins inside `vehicle.Advance()`; "per_step" is the same timer called explicitly; "cumulative" compares totals
A plant with no `ChVehicle` has nothing to spin for it and no `GetRTF()`, so both fall back to doing it from outside

Show Time

- Measured on a laptop, HMMWV + TMeasy tires, step 3 ms (RTF ~ 0.14):
 - "none": drift -69 s after 17 s of wall time
 - "per_step": drift +0.012 s after 17 s and growing
 - "vehicle": drift +0.021 s after 17 s and growing
 - "cumulative": drift +0.000 s
- Drift matters more the closer you are to $RTF = 1$

Variable 1: where the numbers come from

The keyboard, which Chrono reads for you (Lines 535 to 572, 589 to 592)



```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # W/A/S/D drive, 'C' centers steering, 'R' releases the pedals, 'L' locks
    # the inputs. ARROW keys are the chase camera, not driving controls.
    driver = veh.ChInteractiveDriver(vehicle)
    steering_time = 1.0 # time to go from 0 to +1 (or from 0 to -1)
    throttle_time = 1.0 # time to go from 0 to +1
    braking_time = 0.3 # time to go from 0 to +1
    driver.SetSteeringDelta(render_step_size / steering_time)
    driver.SetThrottleDelta(render_step_size / throttle_time)
    driver.SetBrakingDelta(render_step_size / braking_time)
    driver.SetGains(4.0, 4.0, 4.0) # first-order lag from key target to applied input

# ... and then, when the Irrlicht window is built:

vis.AddSkyBox()
plant.attach(vis)
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver) # route Irrlicht key/joystick events to the driver
```

AttachDriver forwards the window's key events - and binds Chrono's gear keys Z X T [] for free
A gamepad or wheel is the same class and one more call: SetInputMode(JOYSTICK), with presets that ship with Chrono
For anything else, nothing is wired up for you - which is the next three slides

Two processes, one UDP contract



Both directions are plain UDP datagrams - two processes, possibly two machines, no shared memory. The return arrow only fires when `SEND_FEEDBACK = True`.

A UDP input source (Lines 219 to 269)

```
class UdpInput:
    """Receive 'steering,throttle,braking[,gear]' datagrams from an operator.
    Non-blocking: the physics loop never waits. No packet since the last step
    means the previous target is held (zero-order hold)."""

    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
        self.operator_addr = None
        self.last = (0.0, 0.0, 0.0)
        self.gear_commands = []

    def poll(self):
        # Drain everything queued and keep only the newest packet. Gear commands
        # are the exception: they accumulate, because dropping one loses a shift.
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
            except BlockingIOError:
                break
            fields = data.decode().split(",")
            if len(fields) < 3:
                continue
            try:
                s, t, b = (float(x) for x in fields[:3])
            except ValueError:
                continue
            self.last = (s, t, b)
            self.operator_addr = addr
            if len(fields) > 3 and fields[3] and fields[3] != "-": # PART 6
                self.gear_commands.append(fields[3][0])
        return self.last
```

Non-blocking socket, drained every step: the newest packet wins, and an empty queue holds the last value
The operator can be anywhere on the network, and the console has no Chrono dependency at all

Sample once, hold, then send state back (Lines 710 to 722, 745 to 755)



```
# PART 4: sample the device ONCE per step and hold it for the step
if device is not None:
    smoother.set_target(*device.poll())
    smoother.advance(step_size)
    smoother.apply_to(driver)
# PART 6: gear commands are events, not levels, so they are applied
# once each rather than held like the three numbers above.
if gearbox is not None:
    for command in device.take_gear_commands():
        gearbox.command_char(command)

# Get driver inputs (three floats) - this is the whole human-in-the-loop contract
driver_inputs = driver.GetInputs()

# ... and at the end of the same step:

# PART 4: feedback to the operator (speed, RTF, lateral acceleration, applied
# inputs, and since PART 6 the gear)
if SEND_FEEDBACK and device is not None and step_number % render_steps == 0:
    acc = (vehicle.GetPointAcceleration(chrono.ChVector3d(0, 0, 0)).y
           if vehicle else 0.0)
    rtf = vehicle.GetRTF() if vehicle else 0.0
    gear = gearbox.describe() if gearbox else plant.status() or "--"
    device.send_feedback(
        f"{sim_time:.3f},{plant.speed():.3f},{rtf:.3f},{acc:.3f},"
        f"{driver_inputs.m_steering:.3f},{driver_inputs.m_throttle:.3f},"
        f"{driver_inputs.m_braking:.3f},{gear}")
```

poll -> smooth -> write, then GetInputs() as before; the physics loop never waits on the device
Feedback goes back at render rate (50 Hz), not every physics step - the human cannot use 333 Hz

Show Time



- Two terminals on one laptop over localhost - or two machines over the network, same code either way
- Stop the console mid-drive: the last input is held - what should a real simulator do here?

Variable 2: what the numbers drive

PLANT: a car, a rover, a crane

- PLANT switches what the same three numbers, from the same devices, are wired into
 - "vehicle" a Chrono::Vehicle - steering, throttle, braking, as usual
 - "rover" a Viper rover - steering becomes a wheel angle in radians, throttle a commanded wheel speed
 - "crane" a gantry crane - steering becomes cross-travel, throttle and braking forward/reverse travel
- The rover shows what "the same three numbers" really costs: braking has no meaning there
 - ViperDriver has no brake input, so holding the brake releases the drive and it coasts. Real limits show up like this
- Neither has a ChVehicle, so ChInteractiveDriver cannot read the keyboard for them - they take UDP, console unchanged

The crane: no Chrono::Vehicle at all (hil_plants.py 319 to 324)

```
def apply(self, inputs):  
    t = self.system.GetChTime()  
    # Throttle drives forward, braking drives back: two pedals, one axis.  
    long_cmd = (inputs.m_throttle - inputs.m_braking) * self.MAX_LONG_SPEED  
    self.long_speed.SetSetpoint(long_cmd, t)  
    self.cross_speed.SetSetpoint(inputs.m_steering * self.MAX_CROSS_SPEED, t)
```

Four rigid bodies, two speed motors and a distance constraint - and that `apply()` is the whole interface to it
The payload swings, with nothing to damp it but the operator, and the console prints the swing angle and the miss distance
Placing it without a swing is a genuinely hard manual task, which is the clearest answer to why a human goes in the loop at all

Show Time



- PLANT = "crane" over udp: put the load on the green pad with the swing under 2 degrees
- Then PLANT = "rover" and hold the brake: it coasts rather than stopping
- Same loop, same console, same three numbers. Only PLANT changed

Variable 3: where it happens

McCity: driving somewhere real

- SCENE = "mcity" swaps the 200 x 200 m patch for a real 32-acre test facility
 - road surface driven as a collision mesh; buildings, poles, signal heads drawn from a placement manifest
 - MCITY_DETAIL = "ground" | "light" | "full" trades scenery for frame rate - start at "ground" on a laptop
 - 860 placements cost 230 meshes: one ChVisualShape, added to a body many times, each time with its own frame
- Third-party and generated, not shipped: a missing dataset falls back to flat terrain with instructions
- Two things that cost time to rediscover:
 - Drive the DRAWN geometry: Mcity's OpenDRIVE elevation differs from the road mesh by -0.24 to +0.29 m (5th/95th pct)
 - Read the spawn height from the mesh: RigidTerrain raycasts a collision system that does not exist yet and answers 0

And the one thing that is not a number at all

```
# A ChDriver carries three numbers, and a gear is not one of them: the
# gearbox belongs to the VEHICLE, so it is reached through the vehicle.

transmission = vehicle.GetTransmission()
transmission.ShiftUp()                # or ShiftDown(), SetGear(n)
transmission.asAutomatic().SetDriveMode(...) # D / N / R
transmission.asAutomatic().SetShiftMode(...) # let it shift, or row it

# On the keyboard you write none of this - vis.AttachDriver(driver) binds:
#   Z  D <-> R      X  neutral      T  AUTO <-> MANUAL    [ ]  down / up
#
# For a device Chrono has never heard of, nothing is wired up. That is the
# point of PART 4: hil_gearbox.Gearbox turns one character off the socket
# into the calls above, and operator_console.py sends it in a 4th field.
```

A steering value is a LEVEL: re-sent every frame, and a lost packet costs nothing

A gear change is an EVENT: a lost one is a shift that never happened - sent once, on the key-down edge, applied exactly once

That distinction is the last thing this tutorial has to teach, and it is not specific to gears or to Chrono

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL): the same three-float interface, scaled up
 - SDL steering wheels with force feedback, cumulative real-time timer, multi-vehicle traffic
 - ROS 2 and Unity bridges, teleoperation over the network, deformable (SCM) terrain
- Chrono::Sensor: cameras/lidar rendered from the vehicle for the operator's screen
- Chrono::Synchrono: several humans in the same world, each on their own machine
- Takeaways
 - Real time in Chrono is soft: spin once per step, and measure the drift
 - The human interface is three floats per step, sampled once and smoothed
 - Never let the input device block the physics loop
 - Levels are held, events are drained - a gear is an event
 - Nothing in the pattern needs a vehicle: swap the plant, keep the loop

Any questions?

Thank you.

kasha2@wisc.edu

Simulation-Based Engineering Lab
University of Wisconsin - Madison

Lab website: <http://sbel.wisc.edu>

Chrono website: <http://www.projectchrono.org>

Source code: <https://github.com/projectchrono>

Tutorial code + slides: <https://github.com/ksha23/chrono-hil-tutorial>